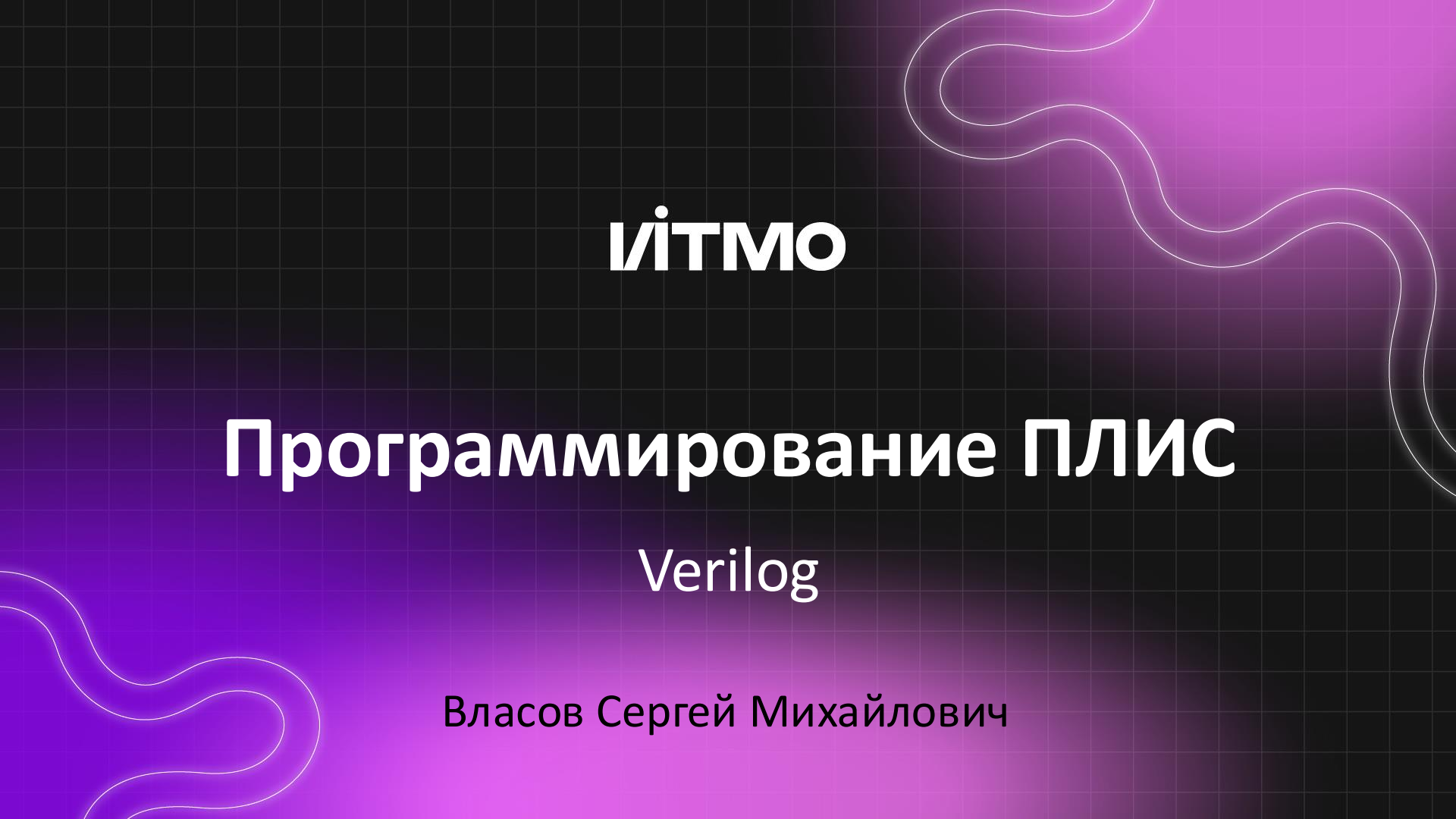




ІІТМО

Программирование ПЛИС

Verilog

Власов Сергей Михайлович

Verilog HDL — Типы данных

Одноразрядные:

- wire,
- reg.



Многоразрядные (веторы):

- `wire[31:0]`,
- `reg[0:31]`.

Объявляется как **[high : low]** или **[low : high]** – little/big endian – левое число всегда старший бит.

Выбор части вектора:

- `vector[N]` – 1 бит;
- `vector[X : Y]` - выбрать все биты с X по Y (в соответствии с endian);
- `vector[Xstart+ : N]` – выбрать N бит начиная с Xstart;

Verilog HDL — Типы данных симуляции

Только для симуляции



- integer – знаковое ≥ 32 бита,
- real – знаковое double,
- time – знаковое, 64 бита. Хранит время симуляции.

Verilog HDL — Знаковость

Два наиболее широко распространённых способы представления отрицательных чисел:



Прямой код:

Двоичное число, состоящее из N битов и записанное в прямом коде, использует наиболее значимый бит для знака, а остальные N-1 бита для записи абсолютного значения этого числа. Если наиболее значимый бит 0, то число положительное. Если наиболее значимый бит 1, то число отрицательное

1 = minus

0 = plus

Sign bit	128	64	32	16	8	4	2	1	
0	0	0	0	1	1	0	1	1	+27
1	0	0	0	1	1	0	1	1	-27

Дополнительный код (two's Complement):



Максимальное положительное значение представлено нулем в наиболее значимом разряде и единицами во всех других разрядах двоичного числа: $01\dots111 = 2^{N-1} - 1$. Максимальное отрицательное значение имеет единицу в наиболее значимом разряде и нули во всех остальных разрядах: $10\dots000 = -2^{N-1}$. Отрицательная единица представлена единицами во всех разрядах двоичного числа: $11\dots111$

Чтобы обратить число в дополнительном коде нужно выполнить операцию дополнения до двух:

$$-A = \bar{A} + 1$$

2's Complement Chart

Binary	Decimal
0111	7
0110	6
0101	5
0100	4
0011	3
0010	2
0001	1
0000	0
1111	-1
1110	-2
1101	-3
1100	-4
1011	-5
1010	-6
1001	-7
1000	-8

Binary	Decimal
00000111	7
00000110	6
00000101	5
00000100	4
00000011	3
00000010	2
00000001	1
00000000	0
11111111	-1
11111110	-2
11111101	-3
11111100	-4
11111011	-5
11111010	-6
11111001	-7
11111000	-8

- Convert 10011001 from binary to decimal:

powers of 2	-2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
decimal weights	-128	64	32	16	8	4	2	1
binary digits	1	0	0	1	1	0	0	1

- 10011001 in decimal is $-128 + 16 + 8 + 1 = -103$

Переменная часть вектора

- **wire, net** – беззнаковые по умолчанию;
- **integer, real** – знаковые.

Ключевое слово **signed/unsigned**:

```
wire signed [31:0] busA, busB, busC;  
reg signed [31:0] data;
```

В числах (**s/S**):

```
4'sd15 // знаковое, отрицательное = -1  
5'sd15 // знаковое, положительное = 15
```

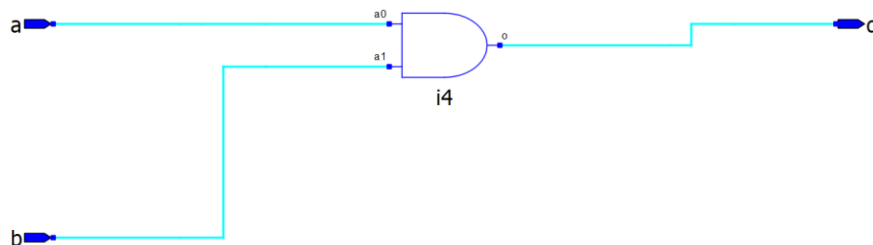
Системные задачи \$signed/ \$unsigned



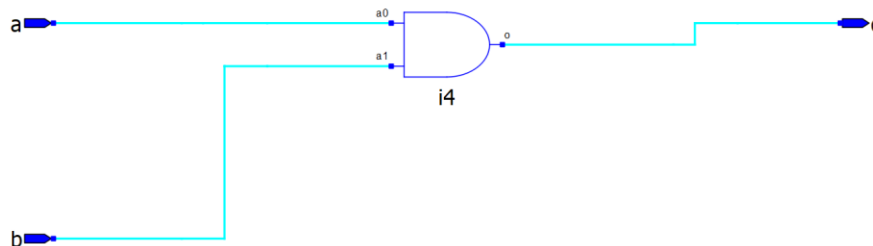
```
wire [4:0] sum;  
wire sign;  
  
reg [4:0] A;  
reg [4:0] B;  
  
assign {sign, sum}= $signed ({1'b0,A}) + B;
```

Verilog HDL — Типы данных

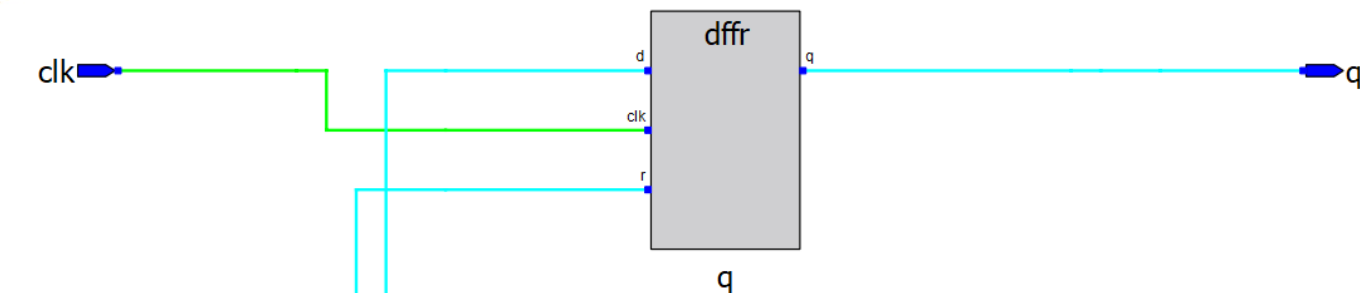
```
1 module top(  
2     input a, b,  
3     output q  
4 );  
5  
6  
7 assign q = a & b;  
8  
9 endmodule  
10
```



```
1 module top(  
2     input a, b,  
3     output reg q  
4 );  
5  
6 always @(*) q = a & b;  
7  
8 endmodule  
9
```

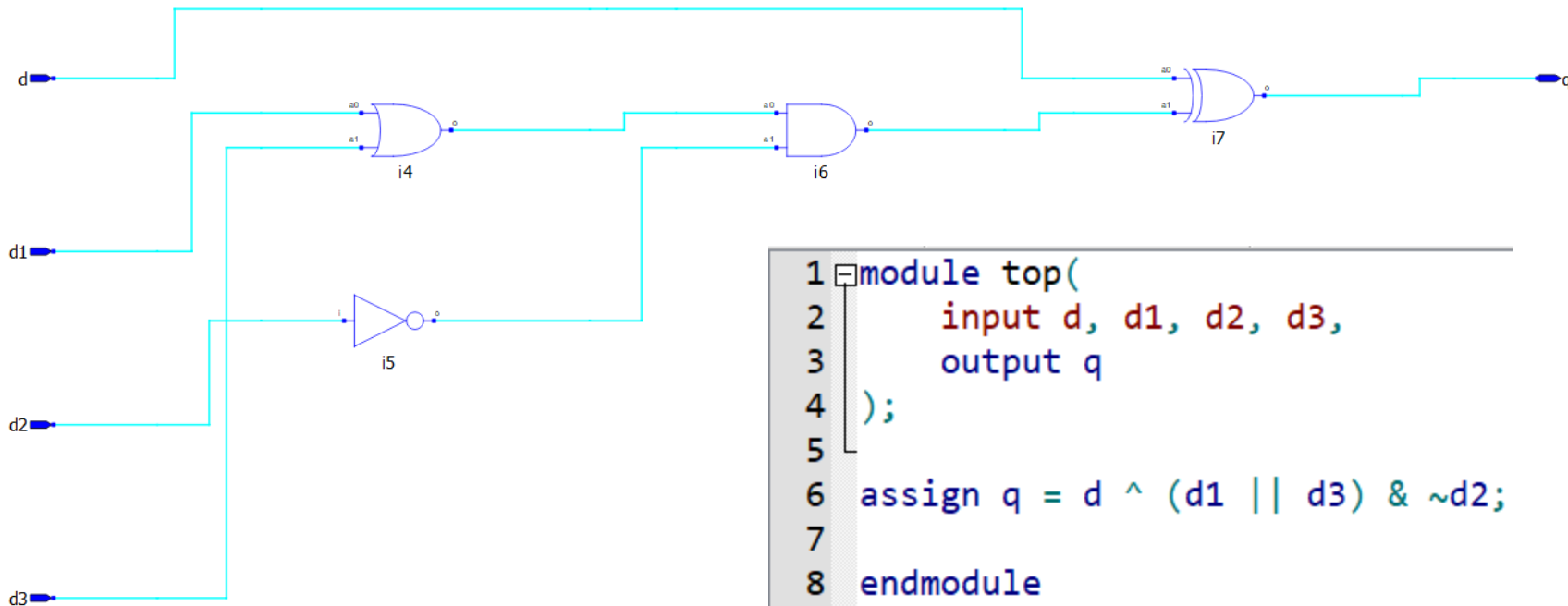


Verilog HDL — Типы данных

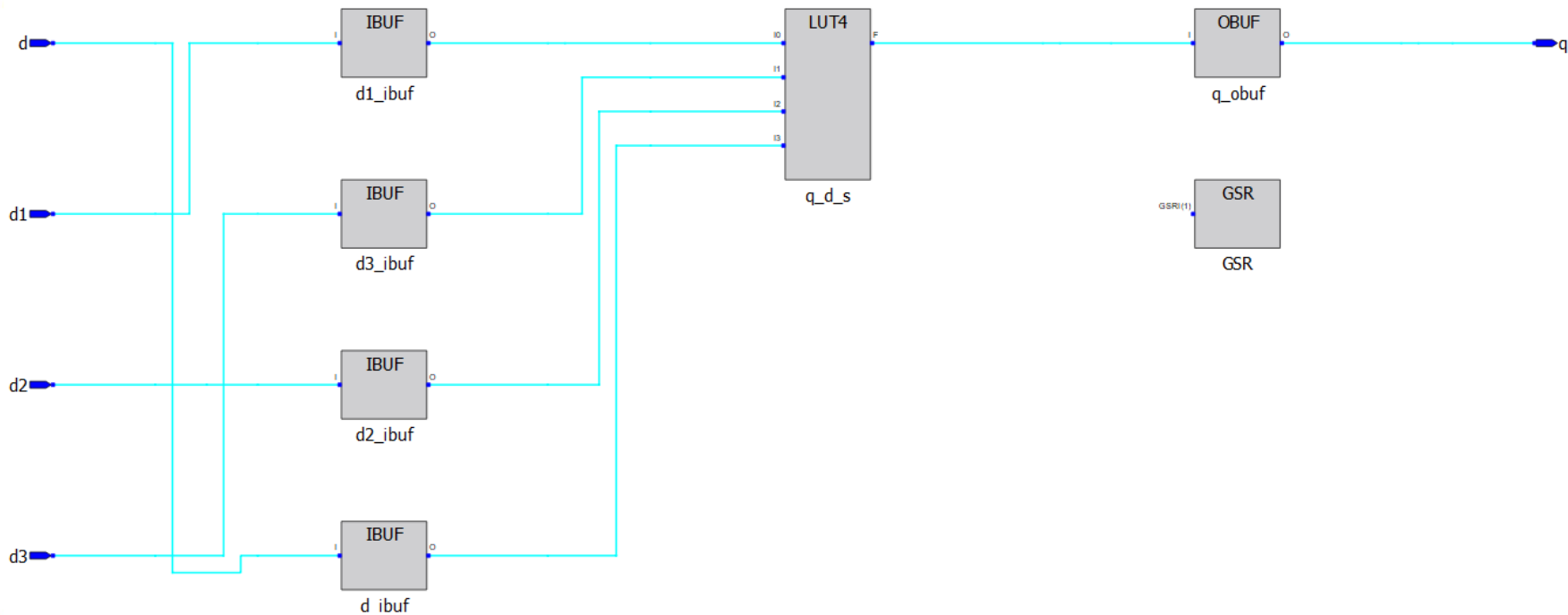


```
1 module top(  
2     input clk, reset, d,  
3     output reg q  
4 );  
5  
6 always @(posedge clk or posedge reset) begin  
7     if (reset) q <= 1'b0;  
8     else q <= d;  
9 end  
10  
11 endmodule  
12
```

Verilog HDL — Типы данных

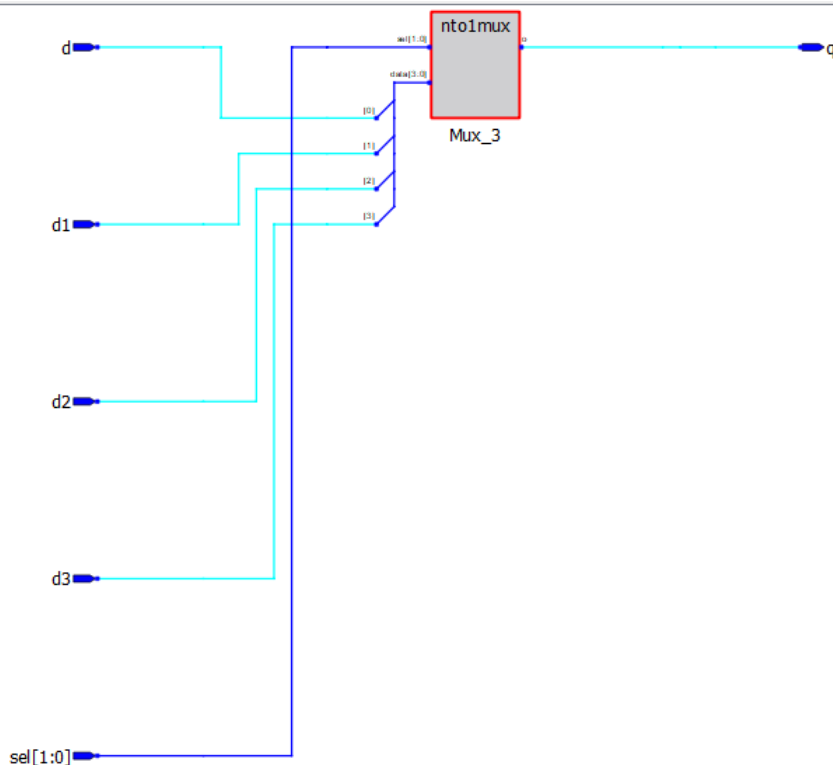


Verilog HDL — Типы данных

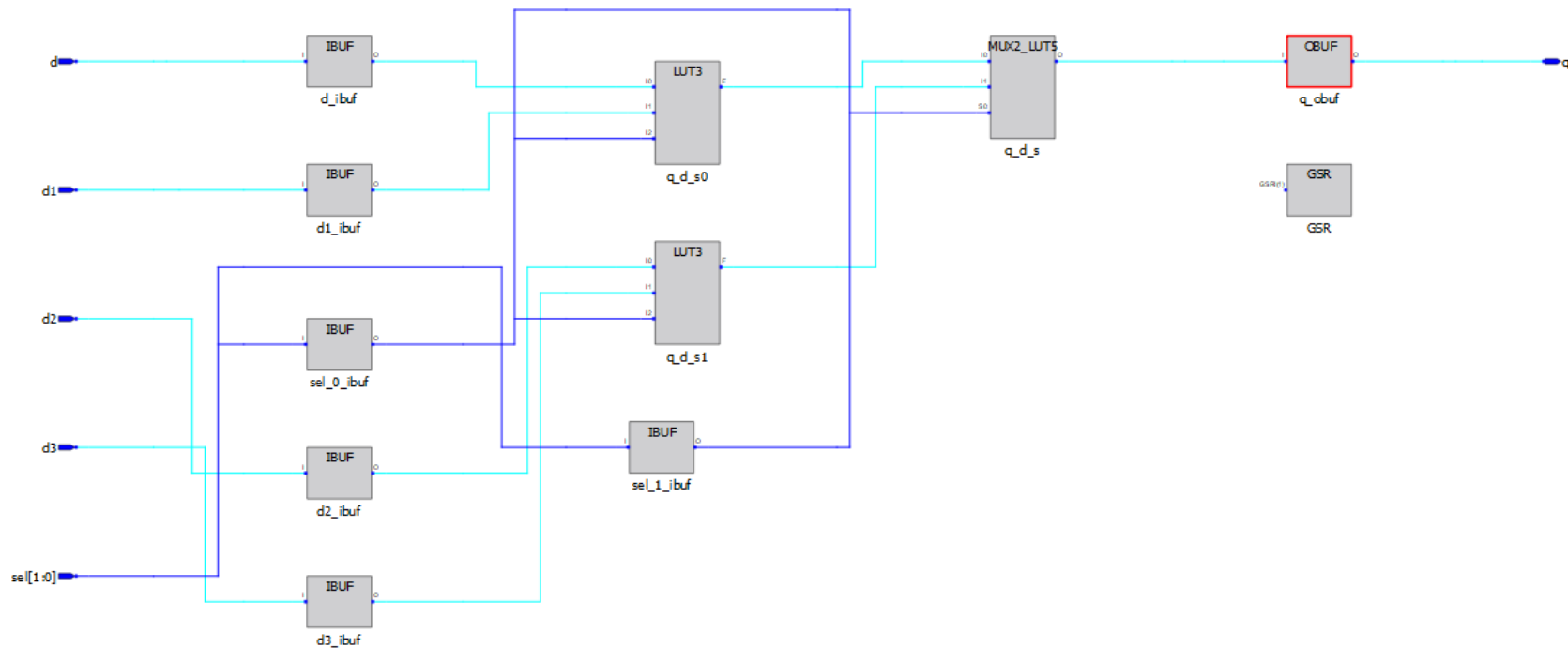


Verilog HDL — Типы данных

```
1 module top(  
2     input d, d1, d2, d3,  
3     input[1:0] sel,  
4     output reg q  
5 );  
6 // mux 4x2  
7 always@(*) begin  
8     case(sel)  
9         2'b00: q <= d;  
10        2'b01: q <= d1;  
11        2'b10: q <= d2;  
12        2'b11: q <= d3;  
13    endcase  
14 end  
15 endmodule  
16  
17  
18
```



Verilog HDL — Типы данных



- Числа могут быть с явным указанием разрядности и без таковой.

- **Размерные:**

<размер> ' <знаковость> <базовый формат> <число>

<размер> — должен быть десятичным числом (количество **бит** (не разрядов!) для хранения значения числа)

<знаковость> — указание того, что данный набор бит (т.е. двоичное число) должен трактоваться как знаковое число (причем отрицательные — как *two's complement*) - (s/S)

<базовый формат> - десятичный (d/D), двоичный (b/B), восьмеричный (o/O), шестнадцатеричный (h/H)

<число> - десятичные *0...9* и буквы *af*

- **Безразмерные:** указывается без разрядности (по умолчанию – 32 бита), например:

' <знаковость> <базовый формат> <число>

<число> // трактуется как integer и знаковое по-умолчанию!



- **Разрядность:**

`4'b1111` // Это 4-битное двоичное число

`12'habc` // Это 12-битное шестнадцатеричное число

`16'd255` // Это 16-битное десятичное число.

`3'b1000` // Ошибка, СЗР будут отброшены и записаны только оставшиеся МЗР! =>
`3'b000`

`5'b1000` // СЗР будет **ВСЕГДА дополнен нулями**, не важно, 1\0 в СЗР самого числа
(даже если значение явно указано как *signed*!) => `5'b01000`

- **Без указания разрядности:**

`23456` // По умолчанию 32-бит знаковое положительное десятичное число

`'hc3` // Это 32-битное шестнадцатеричное число

`'o21` // Это 32-битное восьмеричное число

- **Подсказка:** используйте «`_`» для удобства чтения 'b чисел (Verilog игнорирует «`_`»)

Пример: `16'b1110_0000_0011_1111`



Кроме числовых значений похожим образом можно назначать два других - z (состояние высокого импеданса) и x (неопределенное значение).



Значение x используется для показания недействительного логического уровня. Например если на шину одновременно попадает 0 и 1 с двух активных элементов, то в результате получаем x, что указывает на конфликт.

Состояние z используется для указания на плавающее значение. Использование z-состояния, в частности, полезно для описания буфера с тремя состояниями, состояние выхода которого является плавающим, когда на вход разрешения подан 0.

Verilog HDL — операции и приоритеты

OR



$$Y = A + B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

NOT



$$Y = \bar{A}$$

A	Y
0	1
1	0

AND



$$Y = AB$$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

XOR



$$Y = A \oplus B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

NAND



$$Y = \overline{AB}$$

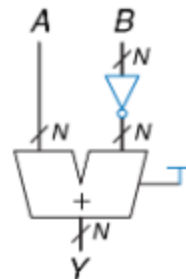
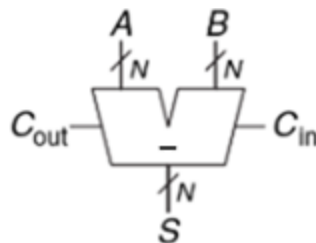
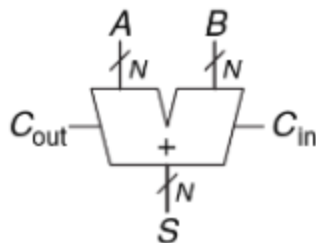
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

NOR

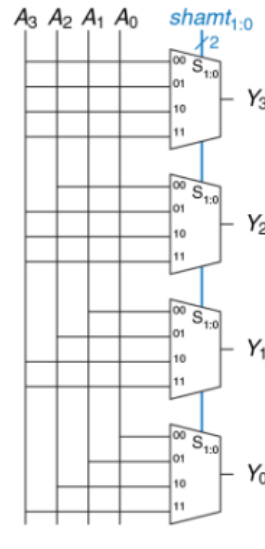
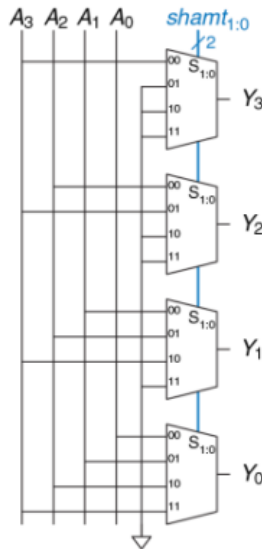
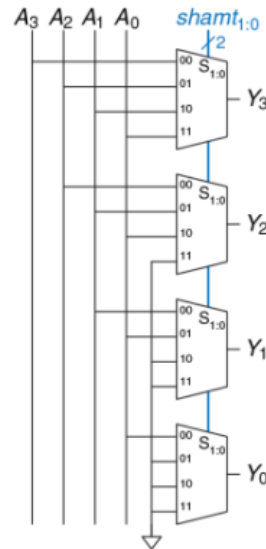
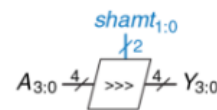
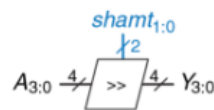
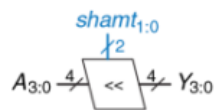
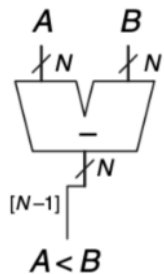
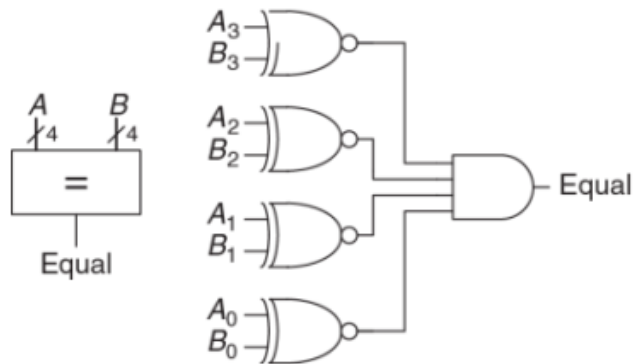


$$Y = \overline{A + B}$$

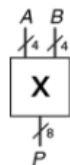
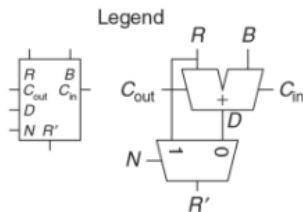
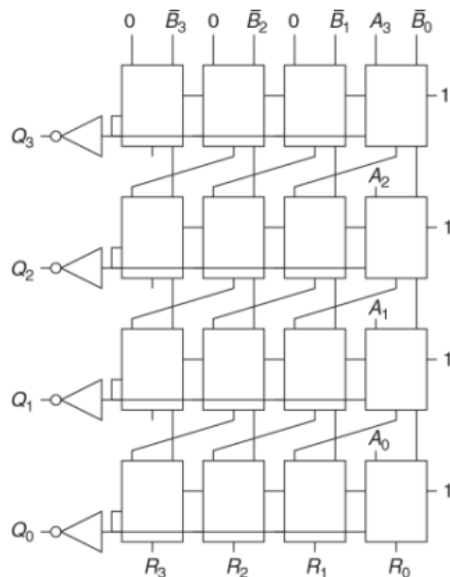
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0



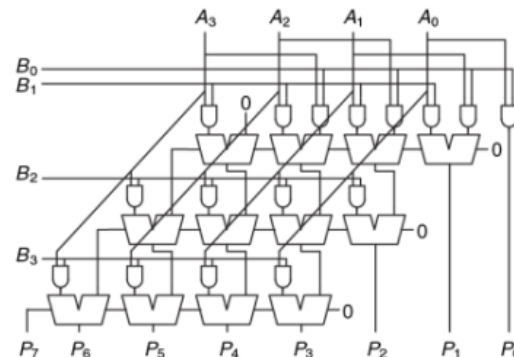
Verilog HDL — операции и приоритеты



Verilog HDL — операции и приоритеты



$$\begin{array}{r}
 \begin{array}{cccc}
 A_3 & A_2 & A_1 & A_0 \\
 \times & B_3 & B_2 & B_1 & B_0 \\
 \hline
 A_3B_0 & A_2B_0 & A_1B_0 & A_0B_0 \\
 A_3B_1 & A_2B_1 & A_1B_1 & A_0B_1 \\
 A_3B_2 & A_2B_2 & A_1B_2 & A_0B_2 \\
 A_3B_3 & A_2B_3 & A_1B_3 & A_0B_3 \\
 \hline
 P_7 & P_6 & P_5 & P_4 & P_3 & P_2 & P_1 & P_0
 \end{array}
 \end{array}$$



Verilog HDL — операции и приоритеты



Операция		Значение
Высший	$\sim$	Побитовое отрицание (НЕ)
	$*, /, \%$	Умножение, деление, остаток
	$+, -$	Сложение, вычитание
	$\ll, \gg$	Сдвиг влево/вправо
	$\lll, \ggg$	Арифметический сдвиг влево/вправо
	$<, <=, >, >=$	Сравнение на больше-меньше

Verilog HDL — операции и приоритеты

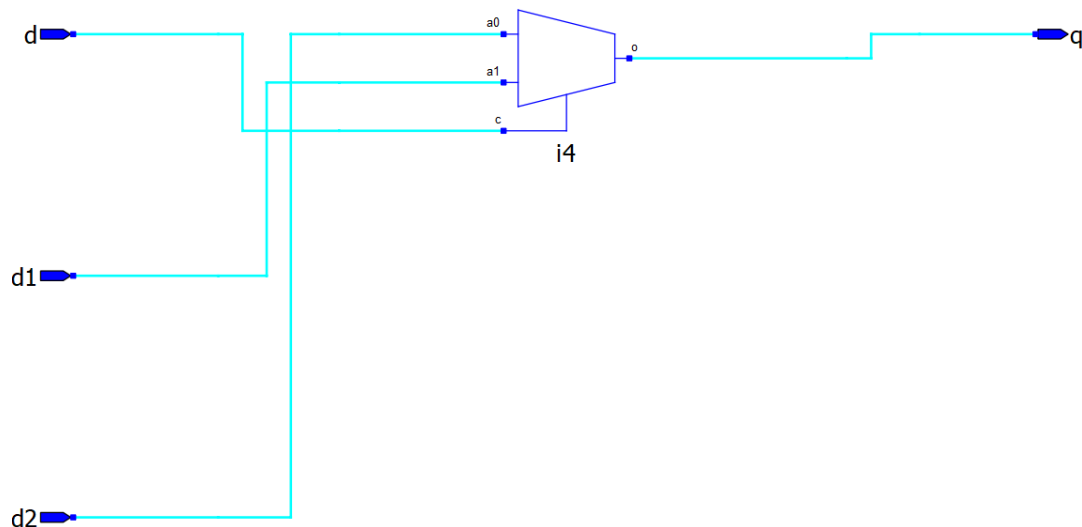


Операция		Значение
Низший	<code>==, !=</code>	Сравнение на равенство
	<code>&, ~&</code>	И, И-НЕ
	<code>^, ~^</code>	Исключающее ИЛИ, исключающее ИЛИ-НЕ
	<code> , ~ </code>	или, или-НЕ
	<code>?:</code>	Условный оператор

Verilog HDL — условный оператор

Условный оператор ?: выбирает между вторым и третьим выражениями, руководствуясь первым выражением, то есть mux2x1

```
module mux2x1(  
    input d1, d2,  
    input d,  
    output q  
);  
    assign q = d ? d1 : d2;  
endmodule
```



Объединения(concatenation)



```
{3'b111, 3'b000} => 6'b111000
{1'b1, 1'b0, 3'b101} => 5'b10101
{4'ha, 4'd10} => 8'b10101010    // 4'ha и 4'd10 в двоичной системе счисления
оба равны 4'b1010
```

```
input [15:0] in;
output [23:0] out;
```

```
assign {out[7:0], out[15:8]} = in; // Меняем местами два байта. Правая и левая
часть — 16-битные векторы.
```

```
assign out[15:0] = {in[7:0], in[15:8]}; // То же самое.
```

```
assign out = {in[7:0], in[15:8]};
```

```
// Тут иначе. 16-битный вектор справа расширен, чтобы соответствовать 24-битному
вектору слева, поэтому out[23:16] равен нулю. В первых двух примерах out[23:16]
не назначены.
```

Повторения (replicator)



```
{5{1'b1}}           // 5'b11111 (или 5'd31 или 5'h1f)
{2{a,b,c}}          // То же что {a,b,c,a,b,c}
{3'd5, {2{3'd6}}}} // 9'b101_110_110. Это объединение 101 со вторым вектором,
// который представляет собой две копии 3'b110.
```

Операторы сокращения

& a[3:0]

// AND: a[3]&a[2]&a[1]&a[0]. Эквивалентно (a[3:0] == 4'hf)

| b[3:0]

// OR: b[3]|b[2]|b[1]|b[0]. Эквивалентно (b[3:0] != 4'h0)

^ c[2:0]

// XOR: c[2]^c[1]^c[0]

Verilog HDL — Массивы

Массивы



```
integer count[8:0]; // Массив из 8 элементов по 32 бита
reg bool[31:0]; // Массив из 32 элементов по 1 биту
reg [4:0] port [0:7]; // Массив из 8 элементов по 5 бит
integer matrix[8:0][0:10]; // Двумерный массив

wire [7:0] 8b_array6 [5:0]; // Массив из 6 элементов по 8 бит
reg [3:0] 4bit;

4bit = 8b_array[0][3:0]; // Выбор 4х бит из нулевого элемента
```



```
module  blackbox // module - ключевое слово, blackbox - имя модуля
(
input  a, b, c // входные порты a,b,c
input  [7:0] bus, // входной порт bus - 8-разрядная шина
output [7:0] bus_out // выходной порт bus_out, также 8-разрядный
);

// Здесь добавляется тело модуля

endmodule // endmodule - конец описания модуля, ключевое слово
```

Подключение модулей

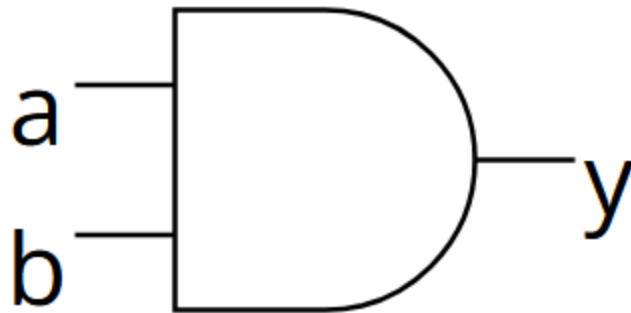
```
module_name instance_name (ports);
```

Подключение портов:

- По положению.
- По имени.

Подключение модулей

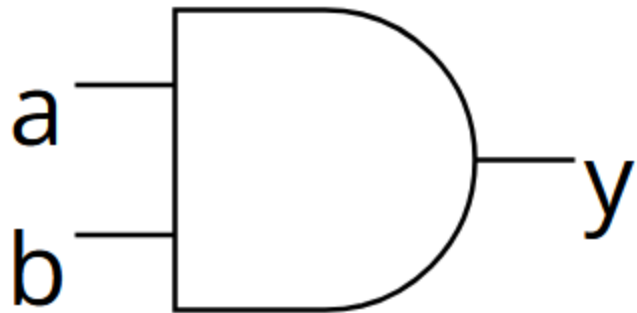
```
module m_and (  
    input a;  
    input b;  
    output y;  
);  
  
    assign y = a & b;  
endmodule
```



Подключение модулей по положению

```
module m_and (input a;input b;output y;);  
    assign y = a & b;  
endmodule
```

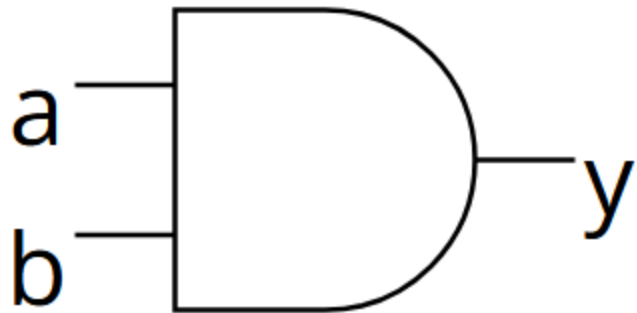
```
module top_module (  
    input in0,in1,  
    output out  
);  
  
m_and m_0(in0,in1,out);  
  
endmodule
```



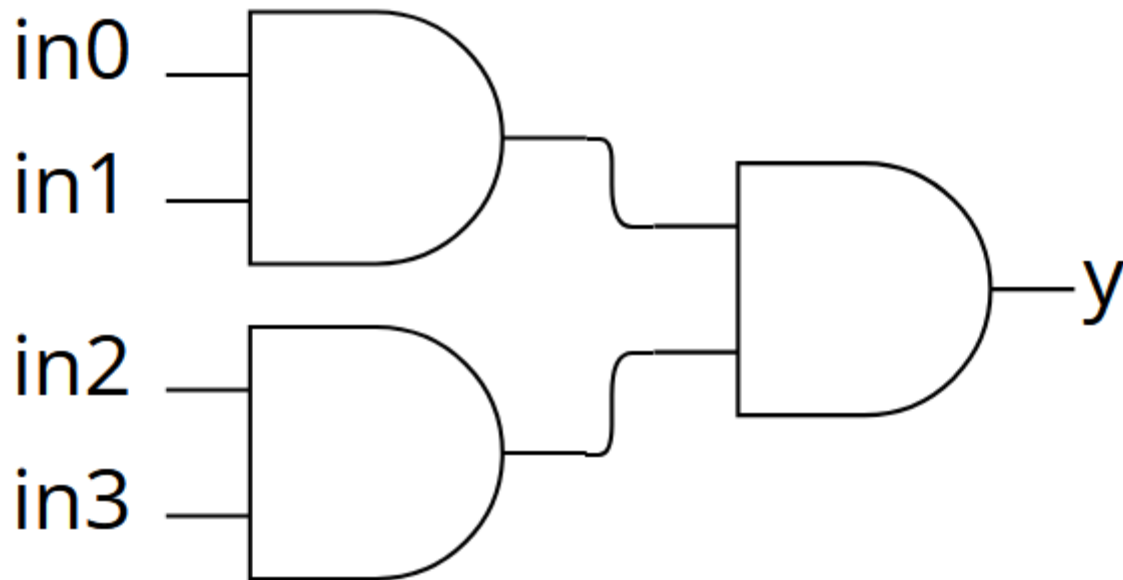
Подключение модулей по имени

```
module m_and (input a;input b;output y;);  
    assign y = a & b;  
endmodule
```

```
module top_module (  
    input in0,in1,  
    output out  
);  
  
m_and m_0(.a(in0), .b(in1), .y(out));  
  
endmodule
```



Подключение модулей

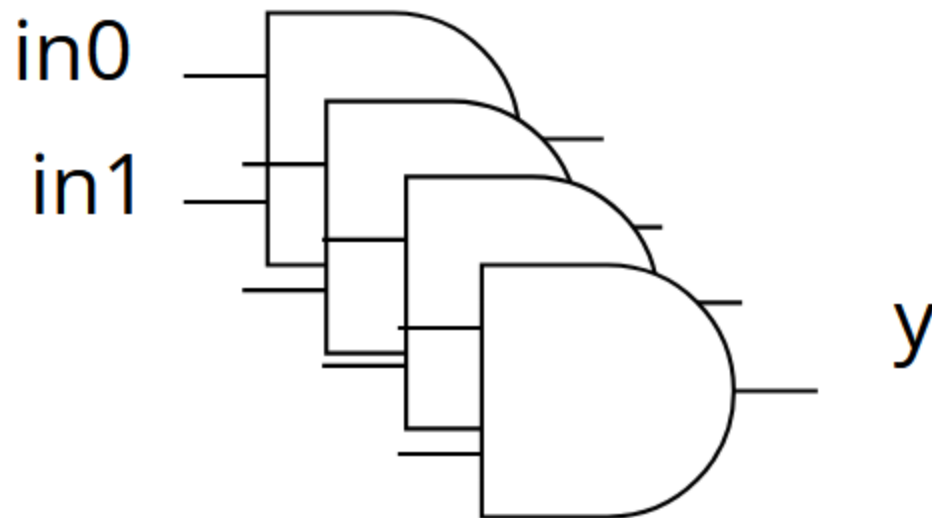


Подключение модулей

```
module top_module (  
    input in0,in1,in2,in3,  
    output out  
);  
  
wire o1,o2;  
  
m_and m_0(.a(in0), .b(in1), .y(o1));  
m_and m_1(.a(in2), .b(in3), .y(o2));  
m_and m_2(.a(o1), .b(o2), .y(out));  
  
endmodule
```



Подключение модулей



Подключение модулей

Массив объектов (array of instances)



```
module top_module (  
    input [3:0]in0,in1,  
    output [3:0]out  
);  
  
m_and m_0[3:0](.a(in0), .b(in1), .y(out));  
  
endmodule
```

Verilog HDL — Параметры

Параметры – константы в пределах модуля



```
module register (q, d, clk, rst_n);  
    parameter SIZE=8;  
    output [SIZE-1:0] q;  
    input [SIZE-1:0] d;  
    input clk, rst_n;  
    reg [SIZE-1:0] q;  
  
    always @(posedge clk or negedge rst_n)  
        if (!rst_n) q <= 0;  
        else q <= d;  
endmodule
```

```
module register2001 #(parameter SIZE=8)
    (output reg [SIZE-1:0] q,
    input [SIZE-1:0] d,
    input clk, rst_n);
    always @(posedge clk, negedge rst_n)
        if (!rst_n) q <= 0;
        else q <= d;
endmodule
```



Переопределение параметра



```
module two_regs1 (q, d, clk, rst_n);  
    output [15:0] q;  
    input [15:0] d;  
    input clk, rst_n;  
    wire [15:0] dx;  
    register #(16) r1(.q(q), .d(dx), .clk(clk), .rst_n(rst_n));  
    register #(16) r2(.q(dx), .d(d), .clk(clk), .rst_n(rst_n));  
endmodule
```

Verilog HDL — Параметры

```
module myreg (q, d, clk, rst_n);  
    parameter Trst = 1,  
        Tckq = 1,  
        SIZE = 4,  
        VERSION = «1.1»;  
    output [SIZE-1:0] q;  
    input  [SIZE-1:0] d;  
    input  clk, rst_n;  
    reg    [SIZE-1:0] q;  
    always @(posedge clk or negedge rst_n)  
        if (!rst_n) q <= #Trst 0;  
        else q <= #Tckq d;  
endmodule
```



```
module bad_wrapper (q, d, clk, rst_n);  
    output [7:0] q;  
    input [7:0] d;  
    input clk, rst_n;  
    // illegal parameter passing example  
    myreg #(:,8) r1 (.q(q), .d(d), .clk(clk), .rst_n(rst_n));  
endmodule
```



```
module good_wrapper (q, d, clk, rst_n);  
    output [7:0] q;  
    input [7:0] d;  
    input clk, rst_n;  
    // первые два параметра должны быть  
    // явно передано, хотя  
    // значение не изменилось  
    myreg #(1,1,8) r1 (.q(q), .d(d), .clk(clk), .rst_n(rst_n));  
endmodule
```

Схема generate

Типы объектов для generate



- модули;
- Gate примитивы;
- assign's;
- Блоки initial и always;
- User-defined примитивы.

Типы данных

- integer, real;
- net, reg;
- time, realtime;
- event;

Схема generate

Конструкции generate



- loop
- if
- case

Не может содержать объявлений

- портов
- parameter
- блоки specify
- блоки specparam

Ключевые слова *generate* и *endgenerate* **могут** использоваться для определения области генерации.

generate loop

genvar – переменная индекса цикла. Должна быть объявлена до использования.



```
genvar k;  
generate  
    for (k = 0; k < 4; k++) begin  
        always@(posedge clk) begin  
            val[k] = a[k] & b[k];  
        end  
    end  
endgenerate
```

generate loop

```
genvar k;  
generate  
    for (k = 0; k < 4; k++) begin  
        always@(posedge clk) begin  
            val[k] = a[k] & b[k];  
        end  
    end  
endgenerate
```

```
always@(posedge clk) begin  
    val[0] = a[0] & b[0];  
    val[1] = a[1] & b[1];  
    val[2] = a[2] & b[2];  
    val[3] = a[3] & b[3];  
end
```

generate loop

```
module gray2bin1 (bin, gray);  
    parameter SIZE = 8; // этот модуль параметризованный  
    output [SIZE-1:0] bin;  
    input [SIZE-1:0] gray;  
    genvar i;  
    generate  
        for (i=0; i<SIZE; i=i+1) begin:bit // блок имеет имя bit  
            assign bin[i] = ^gray[SIZE-1:i];  
            // i ссылается на неявно объявленный localparam чьё  
            // значение в каждом экземпляре блока generate является  
            // значение genvar на момент его разработки.  
        end  
    endgenerate  
endmodule
```

generate loop

```
module addergen1 (co, sum, a, b, ci);
    parameter SIZE = 4;
    output [SIZE-1:0] sum;
    output co;
    input [SIZE-1:0] a, b;
    input ci;
    wire [SIZE :0] c;
    wire [SIZE-1:0] t [1:3];
    genvar i;
    assign c[0] = ci;
    // Иерархические имена экземпляров вентилей:      xor вентили: bit[0].g1 bit[1].g1 bit[2].g1 bit[3].g1
    //                                                    bit[0].g2 bit[1].g2 bit[2].g2 bit[3].g2
    //      and вентили: bit[0].g3 bit[1].g3 bit[2].g3 bit[3].g3
    //                                                    bit[0].g4 bit[1].g4 bit[2].g4 bit[3].g4
    //      or вентили:  bit[0].g5 bit[1].g5 bit[2].g5 bit[3].g5
    // Сгенерированные экземпляры связаны в многомерные сети t[1][3:0] t[2][3:0] t[3][3:0] (Всего 12 сетей)
    for(i=0; i<SIZE; i=i+1) begin:bit
        xor g1 ( t[1][i], a[i], b[i]);
        xor g2 ( sum[i], t[1][i], c[i]);
        and g3 ( t[2][i], a[i], b[i]);
        and g4 ( t[3][i], t[1][i], c[i]);
        or  g5 ( c[i+1], t[2][i], t[3][i]);
    end
    assign co = c[SIZE];
endmodule
```





generate loop

```
module m_and (input a; input b; output y);  
    assign y = a & b;  
endmodule  
  
module my_design  
    #(parameter N=4)  
    (input [N-1:0] in0, in1,  
     output [N-1:0] out);  
  
    genvar i;  
    generate  
        for (i = 0; i<N; i = i + 1) begin  
            m_and m_a(in0[i], in1[i], out[i]); // .a(in[i])  
        end  
    endgenerate  
endmodule
```

generate condition

```
module m_and (input a; input b; output y;);  
    assign y = a & b;  
endmodule
```



```
module m_or (input a; input b; output y;);  
    assign y = a | b;  
endmodule
```

generate condition

```
module gen_if_ex #(parameter sel = 0)(  
    input in0, in1,  
    output out);  
  
    generate  
        if(sel) begin  
            m_and m_and0(in0, in1, out);  
        end  
        else begin  
            m_or m_or0(in0, in1, out);  
        end  
    endgenerate  
endmodule
```



generate condition

```
module gen_if_ex #(parameter sel = 0)(  
    input in0, in1,  
    output out);  
  
    generate  
        case(sel)  
            0:m_and m_and0(in0, in1, out);  
            1:m_or m_or0(in0, in1, out);  
            2:m_xor m_xor0(in0, in1, out);  
            3:m_nand m_nand0(in0, in1, out);  
        endcase  
    endgenerate  
endmodule
```

